

SoC ALSU-Memory System

By

Adel ElSayed Ibrahim

Table of Contents

Executive Summary.....	3
1. Detailed Module Specifications & RTL Mechanics	3
1.1 Rx_UART (UART Receiver).....	3
1.2 Tx_UART (UART Transmitter)	5
1.3 ALSU (Arithmetic Logic Shift Unit)	6
1.4 FSM_ALSU (Master Controller)	7
1.5 RAM (Random Access Memory).....	8
1.6 FSM_Memory (Slave Controller)	9
1.7 SoC_Top (System Integration Wrapper).....	10
Design RTL:	11
1- UART Tx:	11
2- UART Rx.....	15
3- ALSU:	19
4- ALSU FSM:.....	23
5- RAM:	31
6- RAM FSM:.....	32
7- Top Module:.....	37
TEST PLAN:	39
1. Verification Objective	39
2. Testbench Architecture	39
3. Verification Strategy	40
3.1 Randomization.....	40
3.2 FSM Synchronization (Event Waiting)	40
3.3 Golden Model Synchronization.....	40
4. Test Suite and Coverage Plan	41
5. Testbench Tasks (Routines)	42
6. Pass/Fail Criteria	42
7. Testbench:.....	43
SIMULATION:	48
1. Waveform on Questasim:.....	48
2. Transcript:	50

Executive Summary

This document provides a detailed architectural breakdown of the System-on-Chip (SoC) integration between an Advanced Logic Shift Unit (ALSU) Master and a Memory Slave, communicating via a full-duplex UART protocol. The core focus of this specification is the definition, transition conditions, and state diagrams of the four main Finite State Machines (FSMs) driving the system.

1. Detailed Module Specifications & RTL Mechanics

This section provides a structural and functional breakdown of every module within the system, detailing their roles, Input/Output (I/O) ports, and the underlying Register Transfer Level (RTL) mechanics used to implement them.

1.1 Rx_UART (UART Receiver)

Module Overview:

A Serial-In Parallel-Out (SIPO) module designed with noise-resilient sampling mechanics. It decodes incoming UART frames, verifies data integrity (Parity and Stop bits), and extracts the 8-bit parallel payload.

Port Description:

Port Name	Direction	Width	Description
<i>clk, rst</i>	Input	1	Timing and reset signals.
<i>Rx_in</i>	Input	1	Serial receive line.
<i>par_en, par_typ</i>	Input	1	Expected parity configurations for error checking.
<i>prescale</i>	Input	5	Clock divider coefficient for sampling synchronization.
<i>P_data</i>	Output	8	Extracted parallel data payload.
<i>data_valid</i>	Output	1	Asserts high for one clock cycle when a valid frame is decoded.
<i>PARITY_ERROR</i>	Output	1	Asserts high if the calculated parity mismatches the received parity bit.
<i>STP_ERROR</i>	Output	1	Asserts high if a Framing Error occurs (Stop bit is 0).

RTL Implementation Details:

- **Center-Aligned Sampling:** The FSM transitions to *START* upon detecting a falling edge on *Rx_in*. It then waits until *prescale_count* == *half_prescale* (the exact middle of the bit period). If the line is still low, it confirms a valid Start Bit, effectively filtering out high-frequency noise glitches.
- **Data Reconstruction:** During the *DATA* state, the Rx line is sampled precisely at the *half_prescale* mark of every bit width. The sampled value is assigned to the corresponding index of the payload register (*P_data_reg[bit_count] <= Rx_in*).
- **Strict Error Detection:** In the *PARITY* state, the received bit is XOR-compared against the locally computed parity of the reconstructed data. In the *STOP* state, the line is verified to be high. Failures in either check assert the respective error flags, allowing the parent FSM to discard the corrupted transaction.

1.2 Tx_UART (UART Transmitter)

Module Overview:

A Parallel-In Serial-Out (PISO) hardware module. It encapsulates 8-bit parallel data into a standard UART frame (Start bit, Data, Optional Parity, Stop bit) governed by a configurable prescaled baud rate.

Port Description:

Port Name	Direction	Width	Description
<i>clk, rst</i>	Input	1	Timing and reset signals.
<i>data_valid</i>	Input	1	Start pulse to initiate data serialization.
<i>par_en, par_typ</i>	Input	1	Parity generation settings.
<i>prescale</i>	Input	5	Clock divider coefficient to set the baud rate.
<i>P_data</i>	Input	8	Parallel data payload to transmit.
<i>Tx_out</i>	Output	1	Serial transmission line.
<i>Busy</i>	Output	1	High when transmission is actively in progress.

RTL Implementation Details:

- **Integrated Baud Generation:** Instead of relying on an external clock divider, the Tx FSM includes an internal *prescale_count* register. The state machine transitions and bit-shifts only occur when this counter reaches *prescale* - 1.
- **Shift Register Mechanics:** In the *DATA* state, the Least Significant Bit (LSB) is driven to the line ($Tx_out \leq P_data_reg[0]$), while the register simultaneously logical-shifts right ($P_data_reg \leq \{1'b0, P_data_reg[7:1]\}$).
- **Instant Parity Calculation:** Upon detecting the *data_valid* trigger, the module instantly computes the parity bit using a reduction XOR operator ($par_typ ? \sim(^P_data) : (^P_data)$). This pre-calculated bit is stored and shifted out seamlessly during the *PARITY* state.

1.3 ALSU (Arithmetic Logic Shift Unit)

Module Overview:

The ALSU is the core mathematical engine. It executes arithmetic (Add, Multiply), logical (OR, XOR), and bitwise operations (Shift, Rotate). It features advanced capabilities such as logical reduction and operand bypassing. It is a fully synchronous, pipelined unit requiring one cycle to register inputs and one cycle to register outputs.

Port Description:

Port Name	Direction	Width	Description
<i>clk, rst</i>	Input	1	Timing and reset signals.
<i>en</i>	Input	1	Enable signal, driven by the FSM's <i>CALC_1</i> and <i>CALC_2</i> states.
<i>cin</i>	Input	1	Carry-in for the addition operation.
<i>red_op_A/B</i>	Input	1	Flags to enable unary logical reduction on A or B.
<i>bypass_A/B</i>	Input	1	Flags to output A or B directly without processing.
<i>direction</i>	Input	1	Defines the direction for Shift and Rotate operations.
<i>serial_in</i>	Input	1	The bit to be shifted into the accumulator.
<i>opcode</i>	Input	3	Selects the arithmetic/logical operation.
<i>A, B</i>	Input	8	Signed 8-bit operands fetched from memory.
<i>leds</i>	Output	16	Toggles to indicate illegal operations or conflicting configurations.
<i>out</i>	Output	16	Signed 16-bit computed result.

RTL Implementation Details:

- **Continuous Validation Logic:** Continuous assignments (*invalid_red_op*, *invalid_opcode*) aggressively evaluate the configuration inputs. If an illegal state is detected (e.g., applying reduction to an ADD operation, or using opcode 6/7), the *invalid* flag asserts immediately.
- **Fault Indication (LED Toggling):** An *always* block monitors the *invalid* flag. Upon assertion, it inverts the *leds* register (*leds* <= ~*leds*), creating a visual hardware blinking effect to signal a fault condition.
- **Sign Extension & Formatting:** For operations resulting in widths less than 16 bits (like logical OR/XOR or reductions), the RTL uses Verilog concatenation (e.g., {15'b0, |A_reg}) to zero-pad the outputs, ensuring the datapath strictly conforms to the 16-bit output bus requirement.

1.4 FSM_ALSU (Master Controller)

Module Overview:

This is the brain of the system. It houses the Master Finite State Machine (FSM), the UART transceivers, and the *ALSU* datapath. It orchestrates the entire workflow: issuing memory read commands for operands, triggering the arithmetic/logic operations, and issuing memory write commands to store the 16-bit result.

FSM:

Port Description:

Port Name	Direction	Width	Description
<i>clk, rst, EN</i>	Input	1	Core timing and control signals.
<i>Cin .. opcode</i>	Input	-	ALSU configuration inputs routed from the top level.
<i>par_en, par_typ</i>	Input	1	Internal UART configuration parameters.
<i>Rx_in</i>	Input	1	Serial receive line (connected to Slave Tx).
<i>Tx_out</i>	Output	1	Serial transmit line (connected to Slave Rx).
<i>C</i>	Output	16	16-bit computational output to the top level.
<i>leds</i>	Output	16	Error status LEDs to the top level.

RTL Implementation Details:

- **Tx Synchronization (Edge Detection):** The RTL incorporates a *tx_busy_q* register to sample the UART's *tx_busy* signal. This creates a single-cycle *tx_done* pulse (*tx_done* = *tx_busy_q* && !*tx_busy*), ensuring the FSM transitions states exactly when transmission concludes, preventing FSM runaway.
- **Input Capture & Isolation:** Upon detecting *EN* in the *IDLE* state, the FSM captures all control inputs (opcodes, addresses, bypass flags) into internal registers. This protects the datapath from external asynchronous changes during processing.
- **Smart Fetching Optimization:** The FSM evaluates *bypass* and *red_op* flags early. If an operation explicitly only requires operand B, the FSM bypasses the *CMD_READ_A* states entirely, saving significant clock cycles.
- **Data Assembly:** Because the ALSU outputs a 16-bit result (*C*) and the RAM is 8-bit, the FSM decomposes the write operation into two sequential UART transmissions: *DATA_C_LSB* sends *C[7:0]*, followed by *DATA_C_MSB* sending *C[15:8]* to an incremented address.

1.5 RAM (Random Access Memory)

Module Overview:

A parameterized synchronous Block RAM (BRAM) module with a depth of 256 and a width of 8 bits. It serves as the physical storage for operands and final computational results.

Port Description:

Port Name	Direction	Width	Description
<i>clk, rst</i>	Input	1	Timing and reset signals.
<i>en</i>	Input	1	Chip enable; must be high for any memory access.
<i>mem_we</i>	Input	1	Write Enable (Active High).
<i>addr</i>	Input	8	8-bit memory address pointer.
<i>data_in</i>	Input	8	8-bit data payload to be written.
<i>data_out</i>	Output	8	8-bit data read from the specified address.

RTL Implementation Details:

- **Array Inference:** The memory matrix is declared as *reg [DATA_WIDTH-1:0] mem [0:MEM_DEPTH-1]*, allowing synthesis tools to map this directly to physical FPGA Block RAM primitives.
- **Synchronous Access:** Both reads and writes are encapsulated within a single *always @(posedge clk)* block guarded by the chip enable (*en*).
- **Read-During-Write Behavior:** Because the RTL utilizes non-blocking assignments (*<=*), reading from an address simultaneously being written to will yield the *old* memory value. This standard behavior is safely managed by the FSM timings.

1.6 FSM_Memory (Slave Controller)

Module Overview:

The Slave controller continuously monitors the UART Rx line for incoming commands. It acts as an instruction decoder and memory bridge, executing read (*0x0A*) and write (*0x0B*) commands against the integrated synchronous RAM.

Port Description:

Port Name	Direction	Width	Description
<i>clk, rst</i>	Input	1	Core timing and synchronization signals.
<i>Rx_in</i>	Input	1	Serial receive line (commands from Master Tx).
<i>par_en,</i> <i>par_typ</i>	Input	1	UART configuration to match the Master's baud/parity settings.
<i>Tx_out</i>	Output	1	Serial transmit line (sending read data to Master Rx).

RTL Implementation Details:

- **Command Decoding:** In the *WAIT_CMD* state, the FSM evaluates the validated *rx_data*. A value of *0x0A* branches the FSM to address-fetching and memory-reading states, while *0x0B* branches to address-fetching, data-fetching, and memory-writing states.
- **The Read Latency Fix (*READ_WAIT*):** Because the instantiated RAM is synchronous, its output (*ram_data_out*) updates one clock cycle after the read enable (*mem_en*) is asserted. The FSM utilizes a deliberate *READ_WAIT* state to absorb this 1-cycle hardware latency. This guarantees that the UART transmitter is fed stable, valid data rather than outdated memory values.
- **Single-Pulse Tx Trigger:** In the *TX_DATA* state, the transmitter start signal is guarded by *if (!tx_busy && !tx_busy_q) tx_start = 1;*. This strict logic generates a clean, single-cycle pulse, preventing accidental UART re-triggering.

1.7 SoC_Top (System Integration Wrapper)

Module Overview:

The SoC_Top acts as the highest-level wrapper of the design. Its primary function is to integrate the Master node (FSM_ALSU) and the Slave node (FSM_Memory). It establishes the internal communication infrastructure by cross-connecting their UART transmit and receive lines, effectively simulating a full-duplex serial bus on the chip.

Port Description:

Port Name	Direction	Width	Description
clk	Input	1	Global system clock.
rst	Input	1	Global asynchronous active-high reset.
EN	Input	1	Enable pulse to initiate a new processing cycle.
Cin, Serial_in	Input	1	Inputs for ALU addition (carry) and shift operations.
bypass_A, bypass_B	Input	1	Flags to bypass ALSU operations and output A or B directly.
red_op_A, red_op_B	Input	1	Flags to enable logical reduction on operand A or B.
direction	Input	1	Defines shift/rotate direction (0: Left, 1: Right).
location_A, B, C	Input	8	Memory addresses for fetching A and B, and storing the result C.
opcode	Input	3	Operation selector code for the ALSU (0 through 7).
par_en, par_typ	Input	1	UART settings: Parity enable and type (0: Even, 1: Odd).
C	Output	16	The final 16-bit computed result from the ALSU.
leds	Output	16	System status and error indicators (LED 15: UART error, 14-0: ALSU errors).

RTL Implementation Details:

As a strictly structural module, SoC_Top contains no behavioral logic (no always blocks). It declares two internal wires, master_tx_to_slave_rx and slave_tx_to_master_rx, routing the serial data between the instantiated Master and Slave modules. The external physical I/O ports are mapped directly to the Master, while the Slave only receives the clock, reset, and shared UART configuration parameters.

Design RTL:

1- UART Tx:

```
module Tx_UART (  
    input clk, rst, data_valid, par_en, par_typ,  
    input [4:0] prescale,  
    input [7:0] P_data,  
    output reg Tx_out, Busy  
);  
  
    reg [2:0] cs, ns; //STATES  
  
    reg [4:0] prescale_count; //count to send the bits  
  
    reg [7:0] P_data_reg; // Register to serialize the data  
    reg [3:0] bit_count; // Counter to keep track of the number of bits  
transmitted  
    reg parity_bit; // Register to hold the calculated parity bit  
    reg par_en_reg; // Register to hold the parity enable signal  
  
    parameter IDLE = 3'b000, START = 3'b001, DATA = 3'b010, PARITY = 3'b011, STOP  
= 3'b100; //STATE encoding  
  
    //STATE REG  
    always @(posedge clk or posedge rst) begin  
        if (rst)  
            cs <= IDLE;  
        else  
            cs <= ns;  
    end  
  
    //Next State logic  
    always @(*) begin  
        ns = cs;  
  
        case (cs)  
            IDLE: begin //IDLE State wait for the parallel data  
                if (data_valid)
```

```

        ns = START;
    end

    START: begin
        if (prescale_count == prescale - 1) // wait for prescale clks
            ns = DATA;
        end

    DATA: begin
        if (prescale_count == prescale - 1 && bit_count == 7) begin // if
data is completely sent
            if (par_en_reg) //if there is parity bit go to parity if not
go to Stop condition
                ns = PARITY;
            else
                ns = STOP;
            end
        end
    end

    PARITY: begin
        if (prescale_count == prescale - 1)
            ns = STOP;
        end

    STOP: begin
        if (prescale_count == prescale - 1)
            ns = IDLE;
        end
    endcase
end

// output logic
always @(posedge clk or posedge rst) begin
    if (rst) begin
        Tx_out <= 1; // keep Tx out high
        Busy <= 0;
        P_data_reg <= 0;
        bit_count <= 0;
        parity_bit <= 0;
        par_en_reg <= 0;
    end
    else begin
        case (cs)
            IDLE: begin

```

```

        Tx_out <= 1;
        Busy <= 0;
        bit_count <= 0;
        if (data_valid) begin //Start Condition
            //capture the parallel data
            P_data_reg <= P_data;
            par_en_reg <= par_en;
            Busy <= 1;
            parity_bit <= par_typ ? ~(^P_data) : (^P_data); //parity
bit calculation
        end
    end

START: begin
    Tx_out <= 0; // Start bit is low
    if (prescale_count < prescale - 1) //wait for the prescale
pulses
        prescale_count <= prescale_count + 1;
    else
        prescale_count <= 0;
    end

DATA: begin // PISO
    Tx_out <= P_data_reg[0]; //send the least sign bit
    Busy <= 1; // set the busy flag high to prevent any other
data to be sent

    if (prescale_count < prescale - 1)
        prescale_count <= prescale_count + 1;
    else begin
        prescale_count <= 0; // set the count to zero to send the
next bit

        bit_count <= bit_count + 1;
        P_data_reg <= {1'b0, P_data_reg[7:1]}; //shift to next
bit

    end
end

PARITY: begin
    Tx_out <= parity_bit; // sending parity bit
    if (prescale_count < prescale - 1)
        prescale_count <= prescale_count + 1;
    else
        prescale_count <= 0;
    end
end

STOP: begin

```

```

        Tx_out <= 1; // Stop bit is high
        if (prescale_count < prescale - 1)
            prescale_count <= prescale_count + 1;
        else begin
            prescale_count <= 0;
            Busy <= 0; //set the busy high to start capture a new
data
                                end
                                end
                                endcase
                                end
                                end
                                endmodule

```

2- UART Rx

```
module Rx_UART (
    input clk, rst, Rx_in, par_en, par_typ,
    input [4:0] prescale,
    output reg [7:0] P_data,
    output reg data_valid, PARITY_ERROR, STP_ERROR
);
    reg [2:0] cs, ns;
    reg [3:0] bit_count;
    reg [4:0] prescale_count;
    reg [7:0] P_data_reg;

    wire [4:0] half_prescale = prescale/2; //calculate the half prescale to
    capture the bit in this count

    //State Encoding
    parameter IDLE = 3'b000, START = 3'b001, DATA = 3'b010, PARITY = 3'b011, STOP
    = 3'b100;

    // STATE REG
    always @(posedge clk or posedge rst) begin
        if (rst)
            cs <= IDLE;
        else
            cs <= ns;
    end

    //Next State logic
    always @(*) begin
        ns = cs;
        case (cs)
            IDLE: begin //Waiting for start bit which is low
                if (!Rx_in)
                    ns = START;
            end
            START: begin
                if (prescale_count == half_prescale && Rx_in == 1) // if there is
                noise in the start bit return to IDLE State
                    ns = IDLE;
                else if (prescale_count == prescale - 1) // wait the prescale
                pulses
                    ns = DATA;
            end
        endcase
    end
```

```

        DATA: begin
            if (prescale_count == prescale - 1 && bit_count == 7) begin //
if data are complete
                if (par_en) //if parity enabled go to Parity Check if not go
to Stop condition
                    ns = PARITY;
                else
                    ns = STOP;
            end
        end

        PARITY: begin
            if (prescale_count == prescale - 1)
                ns = STOP;
            end

        STOP: begin
            if (prescale_count == prescale - 1)
                ns = IDLE;
            end
            default: ns = IDLE;
        endcase
    end

    // Output logic
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            prescale_count <= 0;
            bit_count <= 0;
            P_data_reg <= 0;
            PARITY_ERROR <= 0;
            STP_ERROR <= 0;
            data_valid <= 0;
        end
        else begin
            data_valid <= 0; //make sure the data_valid is low to complete all
bits
            case (cs)
                IDLE: begin
                    prescale_count <= 0;
                    bit_count <= 0;
                    data_valid <= 0;
                end
            end
        end
    end

```

```

        START: begin
            if (prescale_count == half_prescale && Rx_in == 1'b1) begin
// if there is noise in half of start bit
                prescale_count <= 0;
            end
            else if (prescale_count < prescale - 1) begin
                prescale_count <= prescale_count + 1;
            end
            else begin
                prescale_count <= 0; // End of Start Bit
            end
        end

        DATA: begin
            if (prescale_count < prescale - 1)
                prescale_count <= prescale_count + 1;
            else begin
                prescale_count <= 0;
                bit_count <= bit_count + 1; // start the next bit
            end
            if (prescale_count == half_prescale) begin //capture the bit
in the Half Sampling bit
                P_data_reg[bit_count] <= Rx_in;
            end
        end

        PARITY: begin

            if (prescale_count < prescale - 1)
                prescale_count <= prescale_count + 1;
            else
                prescale_count <= 0;

            if (prescale_count == half_prescale) begin
                if (par_typ == 0)
                    PARITY_ERROR <= ^P_data_reg ^ Rx_in; // Even Parity
Check
                else
                    PARITY_ERROR <= ~(^P_data_reg ^ Rx_in); // Odd
Parity Check
            end
        end
    end

```

```

        STOP: begin
            if (prescale_count < prescale - 1) begin
                prescale_count <= prescale_count + 1'b1;
            end else begin
                prescale_count <= 0;
                if (!STP_ERROR) begin
                    data_valid <= 1;
                    P_data <= P_data_reg;
                end
            end
            if (prescale_count == half_prescale) begin
                STP_ERROR <= (Rx_in == 0); //if there is noise in the
Stop bit
            end
        end
    endcase
end
end
endmodule

```

3- ALSU:

```
module ALSU (
    input clk, cin, rst, red_op_A, red_op_B, bypass_A, bypass_B, direction,
    serial_in, en,
    input [2:0] opcode,
    input signed [7:0] A, B,
    output reg [15:0] leds,
    output reg signed [15:0] out
);
    parameter INPUT_PRIORITY = "A";
    parameter FULL_ADDER = "ON";

    //Registers to capture the Data
    reg red_op_A_reg, red_op_B_reg, bypass_A_reg, bypass_B_reg, direction_reg,
    serial_in_reg;
    reg signed cin_reg;
    reg [2:0] opcode_reg;
    reg signed [7:0] A_reg, B_reg;

    //Invalid Checkers
    wire invalid_red_op = (red_op_A_reg | red_op_B_reg) & (opcode_reg != 0 &&
opcode_reg != 1);
    wire invalid_opcode = (opcode_reg == 6) || (opcode_reg == 7);
    wire invalid = invalid_red_op | invalid_opcode;

    always @(posedge clk or posedge rst) begin
        if (rst) begin
            cin_reg <= 0;
            red_op_B_reg <= 0;
            red_op_A_reg <= 0;
            bypass_B_reg <= 0;
            bypass_A_reg <= 0;
            direction_reg <= 0;
            serial_in_reg <= 0;
            opcode_reg <= 0;
            A_reg <= 0;
            B_reg <= 0;
        end else if (en) begin //capture the data
            cin_reg <= cin;
            red_op_B_reg <= red_op_B;
            red_op_A_reg <= red_op_A;
            bypass_B_reg <= bypass_B;
            bypass_A_reg <= bypass_A;
```

```

        direction_reg <= direction;
        serial_in_reg <= serial_in;
        opcode_reg <= opcode;
        A_reg <= A;
        B_reg <= B;
    end
end

always @(posedge clk or posedge rst) begin
    if (rst)
        leds <= 0;
    else if (en) begin
        if (invalid) //Toggle the Leds
            leds <= ~leds;
        else
            leds <= 0;
    end
end

always @(posedge clk or posedge rst) begin

    if (rst) begin
        out <= 0;
    end else if (en) begin
        if (bypass_A_reg && bypass_B_reg)begin
            if (INPUT_PRIORITY == "A")
                out <= {{8{A_reg[7]}}, A_reg};
            else
                out <= {{8{B_reg[7]}}, B_reg};
        end

        else if (bypass_A_reg)
            out <= {{8{A_reg[7]}}, A_reg};
        else if (bypass_B_reg)
            out <= {{8{B_reg[7]}}, B_reg};
        else if (invalid)
            out <= 0;

        else begin
            case (opcode_reg)

                3'h0: begin // OR
                    if (red_op_A_reg && red_op_B_reg)begin
                        if (INPUT_PRIORITY == "A")

```

```

        out <= {15'b0, |A_reg};
    else
        out <= {15'b0, |B_reg};
    end
    else if (red_op_A_reg)
        out <= {15'b0, |A_reg};
    else if (red_op_B_reg)
        out <= {15'b0, |B_reg};
    else
        out <= {8'b0, (A_reg | B_reg)}; // Zero-extended
logic
end

3'h1: begin // XOR
    if (red_op_A_reg && red_op_B_reg)begin
        if (INPUT_PRIORITY == "A")
            out <= {15'b0, ^A_reg};
        else
            out <= {15'b0, ^B_reg};
        end
    else if (red_op_A_reg)
        out <= {15'b0, ^A_reg};
    else if (red_op_B_reg)
        out <= {15'b0, ^B_reg};
    else
        out <= {8'b0, (A_reg ^ B_reg)}; // Zero-extended
logic
end

3'h2: begin // ADD
    if (FULL_ADDER == "ON")
        out <= A_reg + B_reg + cin_reg;
    else
        out <= A_reg + B_reg;
    end

3'h3: begin // MULT
    out <= A_reg * B_reg;
end

3'h4: begin // SHIFT
    if (direction_reg)
        out <= {out[14:0], serial_in_reg};

```

```

        else
            out <= {serial_in_reg, out[15:1]};
        end

        3'h5: begin // ROTATE
            if (direction_reg)
                out <= {out[14:0], out[15]};
            else
                out <= {out[0], out[15:1]};
            end
        end

        default: out <= 0;
    endcase
end
end
end
endmodule

```

4- ALSU FSM:

```
module FSM_ALSU (
    input clk, EN, rst, Cin, bypass_A, bypass_B, red_op_A, red_op_B, Serial_in,
    direction,
    input [7:0] location_A, location_B, location_C,
    input [2:0] opcode,
    input Rx_in,
    input par_en, par_typ,
    output Tx_out,
    output [15:0] C,
    output [15:0] leds
);

parameter DATA_WIDTH = 8;
parameter LOCATION_WIDTH = 8;

// UART Signals
wire rx_valid;
wire [7:0] rx_data;
wire rx_err, rx_parity_err, rx_stp_err;

assign rx_err = rx_parity_err | rx_stp_err; //error

reg uart_err_flag; // LED 15 Control
wire [15:0] alsu_leds;
assign leds = {uart_err_flag, alsu_leds[14:0]};

wire [4:0] prescale = 8;
wire tx_busy;
reg tx_start;
reg [7:0] tx_data;

// ALSU Signal to start CALC
reg alsu_en;

// States
reg [3:0] cs, ns;

// Register to capture Inputs
reg [LOCATION_WIDTH - 1:0] location_A_reg, location_B_reg, location_C_reg;
reg Cin_reg, bypass_A_reg, bypass_B_reg, red_op_A_reg, red_op_B_reg,
Serial_in_reg, direction_reg;
```

```

reg [2:0] opcode_reg;
reg [DATA_WIDTH - 1:0] A_reg, B_reg;

// capture the negedge of Busy to make sure that Tx is done
reg tx_busy_q;
always @(posedge clk or posedge rst) begin
    if (rst)
        tx_busy_q <= 0;
    else
        tx_busy_q <= tx_busy;
end
wire tx_done;
assign tx_done = (tx_busy_q && !tx_busy);

//ALSU INST
ALSU ALSU_inst (.A(A_reg), .B(B_reg), .cin(Cin_reg),
.serial_in(Serial_in_reg),
.red_op_A(red_op_A_reg), .red_op_B(red_op_B_reg), .opcode(opcode_reg),
.bypass_A(bypass_A_reg), .bypass_B(bypass_B_reg), .en(alsu_en),
.clk(clk), .rst(rst), .direction(direction_reg), .leds(alsu_leds),
.out(C));

//Tx INST
Tx_UART Tx_ALSU (
    .P_data(tx_data),
    .data_valid(tx_start),
    .par_en(par_en),
    .par_typ(par_typ),
    .prescale(prescale),
    .clk(clk),
    .rst(rst),
    .Tx_out(Tx_out),
    .Busy(tx_busy)
);

//Rx INST
Rx_UART Rx_ALSU (
    .clk(clk), .rst(rst), .Rx_in(Rx_in), .par_en(par_en), .par_typ(par_typ),
    .prescale(prescale), .P_data(rx_data), .data_valid(rx_valid),
    .PARITY_ERROR(rx_parity_err), .STP_ERROR(rx_stp_err)
);

```

```

//State encoding
parameter IDLE = 0, CMD_READ_A = 1, ADDR_A = 2, WAIT_A = 3, CMD_READ_B = 4,
ADDR_B = 5, WAIT_B = 6, CALC_1 = 7, CMD_WRITE_C_LSB = 8, ADDR_C_LSB = 9,
DATA_C_LSB = 10, CMD_WRITE_C_MSB = 11, ADDR_C_MSB = 12, DATA_C_MSB = 13, CALC_2 =
14;

//opcode encoding
parameter OR_OP = 3'h0, XOR_OP = 3'h1, ADD_OP = 3'h2, MULT_OP = 3'h3,
SHIFT_OP = 3'h4, ROTATE_OP = 3'h5, INVALID1 = 3'h6, INVALID2 = 3'h7;

// STATE REGISTER
always @(posedge clk or posedge rst) begin
    if (rst) begin
        cs <= IDLE;
    end else begin
        cs <= ns;
    end
end

// Next State logic
always @(*) begin
    ns = cs;
    tx_start = 0;
    tx_data = 0;
    alsu_en = 0;

    case (cs)
        IDLE: begin
            if (EN) begin
                if ((red_op_B || bypass_B) && !red_op_A && !bypass_A) // if A
is not enabled it will take B only
                    ns = CMD_READ_B;
                else
                    ns = CMD_READ_A;
            end
        end
    endcase
end

```

```

end

CMD_READ_A: begin
    tx_data = 8'h0A; //CMD to Mem to Read Data
    if (tx_done)
        ns = ADDR_A;
    else if (!tx_busy) // Wait Tx to be done
        tx_start = 1;
end

ADDR_A: begin
    tx_data = location_A_reg; // send Location of A to MEM
    if (tx_done)
        ns = WAIT_A;
    else if (!tx_busy)
        tx_start = 1;
end

WAIT_A: begin
    if (rx_valid) begin //Wait the Rx process to be done
        if (rx_err)
            ns = IDLE;
        else if (red_op_A_reg || bypass_A_reg) // if B is not
enabled it will take A only
            ns = CALC_1;
        else
            ns = CMD_READ_B;
    end
end

CMD_READ_B: begin
    tx_data = 8'h0A; //CMD to Mem to Read Data
    if (tx_done)
        ns = ADDR_B;
    else if (!tx_busy) // Wait Tx to be done
        tx_start = 1;
end

```

```

ADDR_B: begin
    tx_data = location_B_reg; // send Location of A to MEM
    if (tx_done)
        ns = WAIT_B;
    else if (!tx_busy)
        tx_start = 1;
end

WAIT_B: begin
    if (rx_valid) begin //Wait Rx process to be done
        if (rx_err)
            ns = IDLE;
        else
            ns = CALC_1;
        end
    end
end

CALC_1: begin
    alsu_en = 1; //Enable ALSU To CALC
    ns = CALC_2;
end

CALC_2: begin
    alsu_en = 1; //Enable ALSU To CALC
    ns = CMD_WRITE_C_LSB;
end

CMD_WRITE_C_LSB: begin
    tx_data = 8'h0B; //CMD to MEM to Write Data
    if (tx_done)
        ns = ADDR_C_LSB;
    else if (!tx_busy)
        tx_start = 1;
end

```

```

ADDR_C_LSB: begin
    tx_data = location_C_reg; // sending location of C
    if (tx_done)
        ns = DATA_C_LSB;
    else if (!tx_busy)
        tx_start = 1;
end

DATA_C_LSB: begin // Cause of C is Double Size of Inputs it will
send in two write process
    tx_data = C[7:0]; // sending The Least Sig Byte of C
    if (tx_done)
        ns = CMD_WRITE_C_MSB;
    else if (!tx_busy)
        tx_start = 1;
end

CMD_WRITE_C_MSB: begin
    tx_data = 8'h0B;
    if (tx_done)
        ns = ADDR_C_MSB;
    else if (!tx_busy)
        tx_start = 1;
end

ADDR_C_MSB: begin
    tx_data = location_C_reg + 1'b1; // send the location of the next
index after C location
    if (tx_done)
        ns = DATA_C_MSB;
    else if (!tx_busy)
        tx_start = 1;
end

DATA_C_MSB: begin
    tx_data = C[15:8]; // sending The Most Sig Byte of C
    if (tx_done)
        ns = IDLE;
    else if (!tx_busy)

```

```

        tx_start = 1;
    end

    default: ns = IDLE;
endcase

end

always @(posedge clk or posedge rst) begin
    if (rst) begin
        location_A_reg <= 0;
        location_B_reg <= 0;
        location_C_reg <= 0;
        Cin_reg <= 0;
        bypass_A_reg <= 0;
        bypass_B_reg <= 0;
        red_op_A_reg <= 0;
        red_op_B_reg <= 0;
        Serial_in_reg <= 0;
        direction_reg <= 0;
        opcode_reg <= 0;
        A_reg <= 0;
        B_reg <= 0;
        uart_err_flag <= 0;
    end

    else begin
        if (cs == IDLE && EN) begin // capture the inputs
            location_A_reg <= location_A;
            location_B_reg <= location_B;
            location_C_reg <= location_C;
            Cin_reg <= Cin;
            bypass_A_reg <= bypass_A;
            bypass_B_reg <= bypass_B;
            red_op_A_reg <= red_op_A;
            red_op_B_reg <= red_op_B;
            Serial_in_reg <= Serial_in;
            direction_reg <= direction;
            opcode_reg <= opcode;
            uart_err_flag <= 0;
        end

        if (cs == WAIT_A && rx_valid) begin

```

```
        if (rx_err)
            uart_err_flag <= 1;
        else
            A_reg <= rx_data;
        end

        if (cs == WAIT_B && rx_valid) begin
            if (rx_err)
                uart_err_flag <= 1;
            else
                B_reg <= rx_data;
            end
        end
    end
end
endmodule
```

5- RAM:

```
module RAM (  
    input clk,  
    input rst,  
    input en,  
    input mem_we,  
    input [7:0] addr,  
    input [7:0] data_in,  
    output reg [7:0] data_out  
);  
    parameter DATA_WIDTH = 8;  
    parameter MEM_DEPTH = 256;  
  
    reg [DATA_WIDTH-1:0] mem [0:MEM_DEPTH-1];  
  
    always @(posedge clk or posedge rst) begin  
        if (rst) begin  
            data_out <= 0;  
        end else if (en) begin  
            if (mem_we) begin //if Write process  
                mem[addr] <= data_in;  
            end  
            else  
                data_out <= mem[addr];  
        end  
    end  
endmodule
```

6- RAM FSM:

```
module FSM_Memory (
    input clk,
    input rst,
    input Rx_in,
    input par_en, par_typ,
    output Tx_out
);

    parameter DATA_WIDTH = 8;
    parameter LOCATION_WIDTH = 8;
    // CMDs Encoding
    parameter CMD_READ = 8'h0A;
    parameter CMD_WRITE = 8'h0B;

    // UART Signals
    wire [4:0] prescale = 8;
    wire rx_valid, tx_busy, rx_parity_err, rx_stp_err, rx_err;
    wire [7:0] rx_data;
    assign rx_err = rx_parity_err | rx_stp_err;
    reg tx_start;
    reg [7:0] tx_data;
    reg mem_en;
    reg mem_we;

    //detect the low edge of Busy to make Sure that Tx is done
    reg tx_busy_q;
    always @(posedge clk or posedge rst) begin
        if (rst)
            tx_busy_q <= 0;
        else
            tx_busy_q <= tx_busy;
    end
    wire tx_done = (tx_busy_q && !tx_busy);

    // STATES Encoding
    parameter WAIT_CMD = 0, WAIT_RD_ADDR = 1, READ_MEM = 2, READ_WAIT = 3,
    TX_DATA = 4, WAIT_WR_ADDR = 5, WAIT_WR_DATA = 6, WRITE_MEM = 7;

    reg [2:0] cs, ns;

    //Registers to capture the data
    reg [LOCATION_WIDTH-1:0] ram_addr_reg;
    reg [DATA_WIDTH-1:0] ram_data_in_reg;
```

```

wire [DATA_WIDTH-1:0]    ram_data_out;

//RAM INS
RAM RAM_inst (
    .clk(clk),
    .rst(rst),
    .en(mem_en),
    .mem_we(mem_we),
    .addr(ram_addr_reg),
    .data_in(ram_data_in_reg),
    .data_out(ram_data_out)
);

Rx_UART Rx_MEM (
    .clk(clk),
    .rst(rst),
    .Rx_in(Rx_in),
    .prescale(prescale),
    .par_en(par_en),
    .par_typ(par_typ),
    .P_data(rx_data),
    .data_valid(rx_valid),
    .PARITY_ERROR(rx_parity_err),
    .STP_ERROR(rx_stp_err)
);

Tx_UART Tx_MEM (
    .clk(clk),
    .rst(rst),
    .P_data(tx_data),
    .data_valid(tx_start),
    .par_en(par_en),
    .par_typ(par_typ),
    .prescale(prescale),
    .Tx_out(Tx_out),
    .Busy(tx_busy)
);

//STATE REG
always @(posedge clk or posedge rst) begin
    if (rst)
        cs <= WAIT_CMD;
    else

```

```

        cs <= ns;
    end

    // Next State Logic
    always @(*) begin
        ns = cs; // if there is no transition stay in the current state

        tx_start = 0;
        tx_data = 0;
        mem_we = 0;
        mem_en = 0;

        case (cs)
            WAIT_CMD: begin //waiting for the Rx to complete deserializition
without errors
                if (rx_valid && !rx_err) begin
                    if (rx_data == CMD_READ)
                        ns = WAIT_RD_ADDR;
                    else if (rx_data == CMD_WRITE)
                        ns = WAIT_WR_ADDR;
                end
            end

            WAIT_RD_ADDR: begin
                if (rx_valid) begin
                    if (rx_err)
                        ns = WAIT_CMD;
                    else
                        ns = READ_MEM;
                end
            end

            READ_MEM: begin
                mem_en = 1;
                ns = READ_WAIT;
            end

            READ_WAIT: begin // Waiting the data from RAM
                mem_en = 1;
                ns = TX_DATA;
            end

            TX_DATA: begin //Sending DATA

```

```

        tx_data = ram_data_out;
        if (!tx_busy && !tx_busy_q)
            tx_start = 1; // Start of Tx

        if (tx_done)
            ns = WAIT_CMD;
    end

    WAIT_WR_ADDR: begin
        if (rx_valid) begin
            if (rx_err)
                ns = WAIT_CMD;
            else
                ns = WAIT_WR_DATA;
        end
    end

    WAIT_WR_DATA: begin
        if (rx_valid) begin
            if (rx_err)
                ns = WAIT_CMD;
            else
                ns = WRITE_MEM;
        end
    end

    WRITE_MEM: begin
        mem_en = 1;
        mem_we = 1;
        ns = WAIT_CMD;
    end

    default: ns = WAIT_CMD;
endcase
end

// output logic
always @(posedge clk or posedge rst) begin
    if (rst) begin
        ram_addr_reg <= 0;
        ram_data_in_reg <= 0;
    end else begin
        if ((cs == WAIT_RD_ADDR || cs == WAIT_WR_ADDR) && rx_valid &&
!rx_err)

```

```
        ram_addr_reg <= rx_data; //capture the data address

    if (cs == WAIT_WR_DATA && rx_valid && !rx_err)
        ram_data_in_reg <= rx_data; // capture the data
    end
end
endmodule
```

7- Top Module:

```
module SoC_Top (  
    input clk,  
    input rst,  
    input EN,  
    input Cin,  
    input bypass_A, bypass_B,  
    input red_op_A, red_op_B,  
    input Serial_in, direction,  
    input [7:0] location_A, location_B, location_C,  
    input [2:0] opcode,  
    input par_en, par_typ,  
    output [15:0] C,  
    output [15:0] leds  
);  
  
    // wires to connect the two IPs  
    wire master_tx_to_slave_rx;  
    wire slave_tx_to_master_rx;  
  
    FSM_ALSU Master (  
        .clk(clk),  
        .EN(EN),  
        .rst(rst),  
        .Cin(Cin),  
        .bypass_A(bypass_A),  
        .bypass_B(bypass_B),  
        .red_op_A(red_op_A),  
        .red_op_B(red_op_B),  
        .Serial_in(Serial_in),  
        .direction(direction),  
        .location_A(location_A),  
        .location_B(location_B),  
        .location_C(location_C),  
        .opcode(opcode),  
        .par_en(par_en),  
        .par_typ(par_typ),  
        .Rx_in(slave_tx_to_master_rx),  
        .Tx_out(master_tx_to_slave_rx),  
        .C(C),  
        .leds(leds)  
    );  
  
    FSM_Memory Slave (  
        .clk(clk),
```

```
.rst(rst),  
.par_en(par_en),  
.par_typ(par_typ),  
.Rx_in(master_tx_to_slave_rx),  
.Tx_out(slave_tx_to_master_rx)  
);  
endmodule
```

TEST PLAN:

1. Verification Objective

The primary objective of this verification plan is to ensure the functional correctness of the SoC_Top module, which integrates the Master FSM (ALSU controller) and the Slave FSM (Memory controller) via UART. The testbench validates that the system correctly fetches operands from memory, performs the requested Arithmetic Logic Shift Unit (ALSU) operations, and accurately writes the 16-bit result back to the designated memory locations.

2. Testbench Architecture

The testbench (TB) utilizes a **Golden Model Verification Strategy**, comparing the RTL output against a known-good reference model.

- **Device Under Test (DUT):** The SoC_Top module containing the FSMs, UART transceivers, ALSU, and RAM.
- **Golden Model:** A standalone ALSU module instantiated directly within the testbench (ALSU_golden). It is driven by the exact same inputs as the DUT.
- **Memory Probing (Backdoor Read):** Instead of waiting for UART transmission to verify the result, the testbench reads the written 16-bit result directly from the Slave RAM hierarchy using a continuous assignment:

```
assign C_in_MEM = {dut.Slave.RAM_inst.mem[location_C + 1],  
dut.Slave.RAM_inst.mem[location_C]};
```
- **Clock Generation:** A 50 MHz clock is generated with a period of 2 time units (forever #1 clk = !clk;).

3. Verification Strategy

3.1 Randomization

To ensure robust memory addressing, the testbench utilizes pseudo-random generation for memory pointers. Before every test case, `location_A`, `location_B`, and `location_C` are randomized across the full address space (0 to 255) using `$urandom_range(0, 255)`.

3.2 FSM Synchronization (Event Waiting)

The test suite heavily relies on FSM state monitoring to advance the simulation time safely. The sequence is as follows:

1. Drive EN high for one clock cycle.
2. Wait for both Master and Slave FSMs to leave the IDLE state (`wait(dut.Master.cs != 0)`).
3. Wait for both FSMs to completely finish the transaction and return to IDLE (`wait(dut.Master.cs == 0)`).
4. Allow 2 extra clock cycles for data stability before triggering the checker.

3.3 Golden Model Synchronization

The ALSU Golden model requires 2 clock cycles to process data (due to the registered inputs and registered outputs in its RTL design). The `setup_inputs` task drives the `alsu_en_reg` high for exactly two negative clock edges to mimic the exact pipeline behavior of the FSM and capture a reliable `C_expected`.

4. Test Suite and Coverage Plan

The test plan covers standard arithmetic, logic, reduction, shift, rotation, bypass, and error-handling capabilities of the ALSU.

Test Case Name	Opcode	bypass_A	bypass_B	red_op_A	red_op_B	Cin	Dir / S_in	Expected Behavior
OR	3'h0	0	0	0	0	0	0 / 0	Performs bitwise OR.
XOR	3'h1	0	0	0	0	0	0 / 0	Performs bitwise XOR.
ADD	3'h2	0	0	0	0	0	0 / 0	Addition: $\$C = A + B\$$.
ADD Cin	3'h2	0	0	0	0	1	0 / 0	Addition with carry: $\$C = A + B + 1\$$.
MULT	3'h3	0	0	0	0	0	0 / 0	Multiplication: $\$C = A \text{ times } B\$$.
OR Red_A	3'h0	0	0	1	0	0	0 / 0	Unary reduction OR on operand A.
XOR Red_B	3'h1	0	0	0	1	0	0 / 0	Unary reduction XOR on operand B.
Bypass A	3'h2	1	0	0	0	0	0 / 0	Ignores opcode, outputs A.
Bypass A,B	3'h3	1	1	0	0	0	0 / 0	Priority logic test (Outputs A based on config).
Shift Left	3'h4	0	0	0	0	0	1 / 1	Shifts left by 1, inserts Serial_in (1).
Rotate Rgt	3'h5	0	0	0	0	0	0 / 0	Rotates accumulator right.
Invalid Op	3'h6	0	0	0	0	0	0 / 0	Triggers invalid flag and LED blink sequence.

5. Testbench Tasks (Routines)

- `assert_reset`: Applies a synchronous reset to all system components, clears all input registers, and initializes the memory payload via `$readmemb("MEM.txt", ...)`.
- `setup_inputs`: Backdoor-reads the values of A and B from the RAM based on randomized locations, feeds them to the Golden Model ALSU alongside control signals, and strobes the Golden Model enable signal to compute the expected result.
- `run_test_case`: Sets the control signals via task arguments, calls `setup_inputs`, generates the EN pulse to start the DUT, and synchronizes with the DUT's FSMs before calling the checker.
- `Check_result`: The automated checker. It compares `C_in_MEM` against `C_expected`. It increments `correct_count` upon success or `error_count` upon failure, logging detailed debug data to the transcript.

6. Pass/Fail Criteria

1. **Data Integrity**: The 16-bit word constructed from `dut.Slave.RAM_inst.mem[location_C+1]` and `[location_C]` must perfectly match the `C_expected` output of the Golden Model for every transaction.
2. **Status Indicators (LEDs)**: The `leds` output array must strictly match `leds_expected`. Specifically, when an invalid opcode (3'h6) is injected, the test verifies that the LEDs accurately toggle.
3. **Completion**: The simulation must report Total Failed : 0 at the end of the sequence. Any non-zero error count constitutes a system failure requiring RTL debugging.

7. Testbench:

```
module TB;
    logic clk, rst, EN, Cin, bypass_A, bypass_B, red_op_A, red_op_B, Serial_in,
direction;
    logic [7:0] location_A, location_B, location_C;
    logic [2:0] opcode;
    logic par_en, par_typ;
    logic [15:0] C;
    logic [15:0] leds;

    logic Cin_reg, bypass_A_reg, bypass_B_reg, red_op_A_reg, red_op_B_reg,
Serial_in_reg, direction_reg;
    logic [2:0] opcode_reg;
    logic [7:0] A, B;
    logic alsu_en_reg;
    logic [15:0] C_expected;
    logic [15:0] leds_expected;

    // C in the location at mem
    logic [15:0] C_in_MEM;

    assign C_in_MEM = {dut.Slave.RAM_inst.mem [location_C +
1],dut.Slave.RAM_inst.mem [location_C]};

    integer correct_count, error_count;

    SoC_Top dut (.clk(clk), .rst(rst), .EN(EN), .Cin(Cin), .bypass_A(bypass_A),
.bypass_B(bypass_B), .red_op_A(red_op_A), .red_op_B(red_op_B),
    .Serial_in(Serial_in), .direction(direction), .location_A(location_A),
.location_B(location_B), .location_C(location_C), .opcode(opcode),
    .par_en(par_en), .par_typ(par_typ), .C(C), .leds(leds)
    );

    ALSU ALSU_golden(.A(A), .B(B), .cin(Cin_reg), .serial_in(Serial_in_reg),
.red_op_A(red_op_A_reg), .red_op_B(red_op_B_reg),
    .opcode(opcode_reg), .bypass_A(bypass_A_reg),
.bypass_B(bypass_B_reg), .clk(clk), .rst(rst), .direction(direction_reg),
    .en(alsu_en_reg), .leds(leds_expected), .out(C_expected));

    initial begin
        clk = 0;
        forever #1 clk = !clk;
    end
end
```

```

initial begin
    correct_count = 0; error_count = 0;
    assert_reset();

    location_A = $urandom_range(0, 255); location_B = $urandom_range(0, 255);
location_C = $urandom_range(0, 255);
    run_test_case("OR", 0, 0, 0, 0, 0, 0, 0, 0);

    location_A = $urandom_range(0, 255); location_B = $urandom_range(0, 255);
location_C = $urandom_range(0, 255);
    run_test_case("XOR", 1, 0, 0, 0, 0, 0, 0, 0);

    location_A = $urandom_range(0, 255); location_B = $urandom_range(0, 255);
location_C = $urandom_range(0, 255);
    run_test_case("ADD", 2, 0, 0, 0, 0, 0, 0, 0);

    location_A = $urandom_range(0, 255); location_B = $urandom_range(0, 255);
location_C = $urandom_range(0, 255);
    run_test_case("ADD Cin", 2, 0, 0, 0, 0, 0, 1, 0);

    location_A = $urandom_range(0, 255); location_B = $urandom_range(0, 255);
location_C = $urandom_range(0, 255);
    run_test_case("MULT", 3, 0, 0, 0, 0, 0, 0, 0);

    location_A = $urandom_range(0, 255); location_B = $urandom_range(0, 255);
location_C = $urandom_range(0, 255);
    run_test_case("OR Red_A ", 3'h0, 0, 0, 1, 0, 0, 0, 0);

    location_A = $urandom_range(0, 255); location_B = $urandom_range(0, 255);
location_C = $urandom_range(0, 255);
    run_test_case("XOR Red_B ", 3'h1, 0, 0, 0, 1, 0, 0, 0);

    location_A = $urandom_range(0, 255); location_B = $urandom_range(0, 255);
location_C = $urandom_range(0, 255);
    run_test_case("Bypass A ", 3'h2, 1, 0, 0, 0, 0, 0, 0);

    location_A = $urandom_range(0, 255); location_B = $urandom_range(0, 255);
location_C = $urandom_range(0, 255);
    run_test_case("Bypass A,B", 3'h3, 1, 1, 0, 0, 0, 0, 0);

    location_A = $urandom_range(0, 255); location_B = $urandom_range(0, 255);
location_C = $urandom_range(0, 255);
    run_test_case("Shift Left", 3'h4, 0, 0, 0, 0, 0, 1, 1);

```

```

        location_A = $urandom_range(0, 255); location_B = $urandom_range(0, 255);
location_C = $urandom_range(0, 255);
        run_test_case("Rotate Rgt", 3'h5, 0, 0, 0, 0, 0, 0, 0);

        location_A = $urandom_range(0, 255); location_B = $urandom_range(0, 255);
location_C = $urandom_range(0, 255);
        run_test_case("Invalid Op", 3'h6, 0, 0, 0, 0, 0, 0, 0);

        if (leds === leds_expected)
            $display("Invalid Opcode toggled LEDs matching Golden Model.");
        else
            $display("[FAIL] Invalid Opcode LEDs mismatch!");

        $display("Total Passed : %0d", correct_count);
        $display("Total Failed : %0d", error_count);
        $stop;
    end

    task assert_reset;
        rst = 1;
        EN = 0;
        Cin = 0;
        bypass_A = 0;
        bypass_B = 0;
        red_op_A = 0;
        red_op_B = 0;
        Serial_in = 0;
        direction = 0;
        par_en = 0;
        par_typ = 0;
        location_A = 0;
        location_B = 1;
        location_C = 2;
        opcode = 0;

        //initailization of RAM
        $readmemb("MEM.txt", dut.Slave.RAM_inst.mem);

        repeat (20) @(negedge clk);
        rst = 0;
        repeat (20) @(negedge clk);
    endtask

```

```

    task setup_inputs; //capture data and make calc on the data in the golden
model
    Cin_reg = Cin;
    bypass_A_reg = bypass_A;
    bypass_B_reg = bypass_B;
    red_op_A_reg = red_op_A;
    red_op_B_reg = red_op_B;
    Serial_in_reg = Serial_in;
    direction_reg = direction;
    opcode_reg = opcode;

    A = dut.Slave.RAM_inst.mem [location_A];
    B = dut.Slave.RAM_inst.mem [location_B];

    alsu_en_reg = 1;
    @(negedge clk);
    @(negedge clk); // PIPELINE BUG FIXED: Added 2nd edge to allow ALSU to
compute
    alsu_en_reg = 0;
endtask

task Check_result(input string test_name);
begin
    if (C_in_MEM != C_expected) begin
        error_count = error_count + 1;
        $display("FAIL %s ERROR: A=%0h, B=%0h, Op=%0b, C_in_mem=%0h,
C_Exp=%0h", test_name, A, B, opcode_reg, C_in_MEM, C_expected);
    end else begin
        correct_count = correct_count + 1;
        $display("PASS %s A=%0h, B=%0h, Op=%0b, C_in_mem=%0h, C_Exp=%0h",
test_name, A, B, opcode_reg, C_in_MEM, C_expected);
    end
end
endtask

task run_test_case(input string name, input [2:0] op, input logic bp_A, bp_B,
r_A, r_B, c_in, s_in, dir);
    opcode = op;
    bypass_A = bp_A;
    bypass_B = bp_B;
    red_op_A = r_A;
    red_op_B = r_B;
    Cin = c_in;
    Serial_in = s_in;

```

```

        direction = dir;

        setup_inputs();

        @(negedge clk);
        EN = 1;
        @(negedge clk);
        EN = 0;

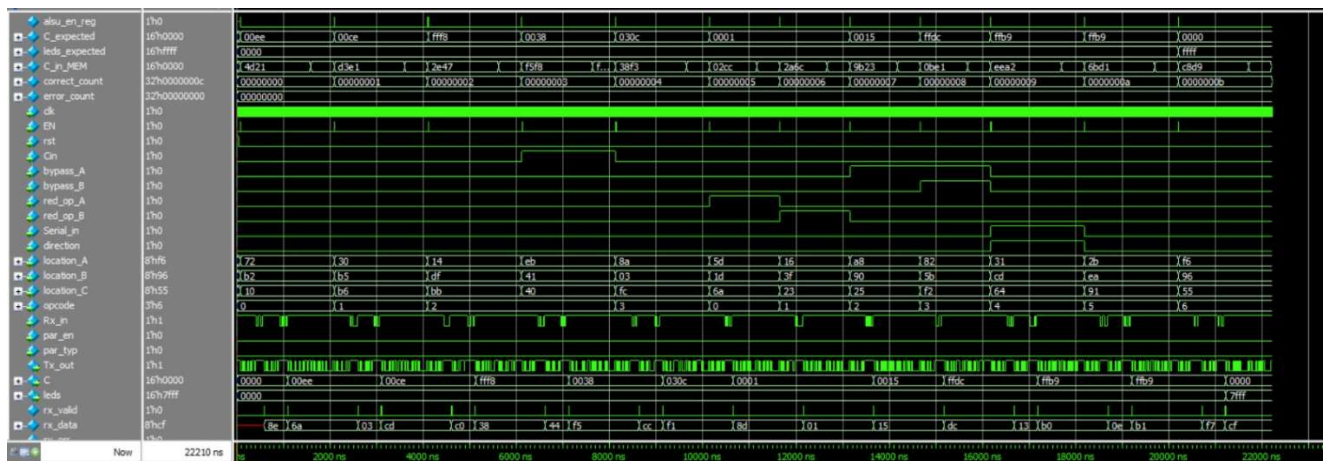
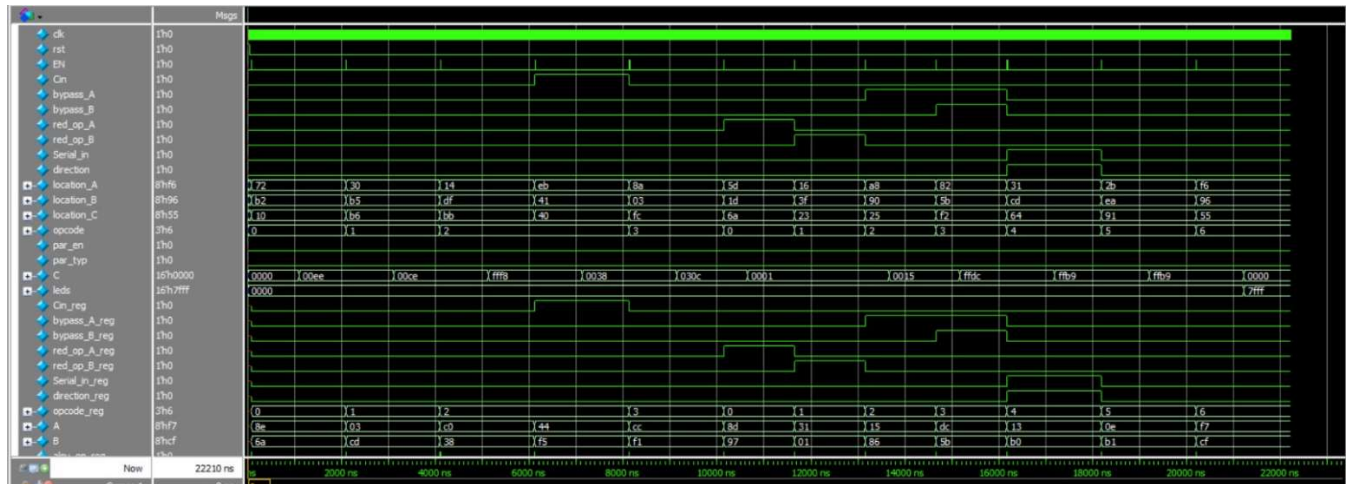
        wait(dut.Master.cs != 0);
        wait (dut.Slave.cs != 0);
        wait(dut.Master.cs == 0);
        wait (dut.Slave.cs == 0);

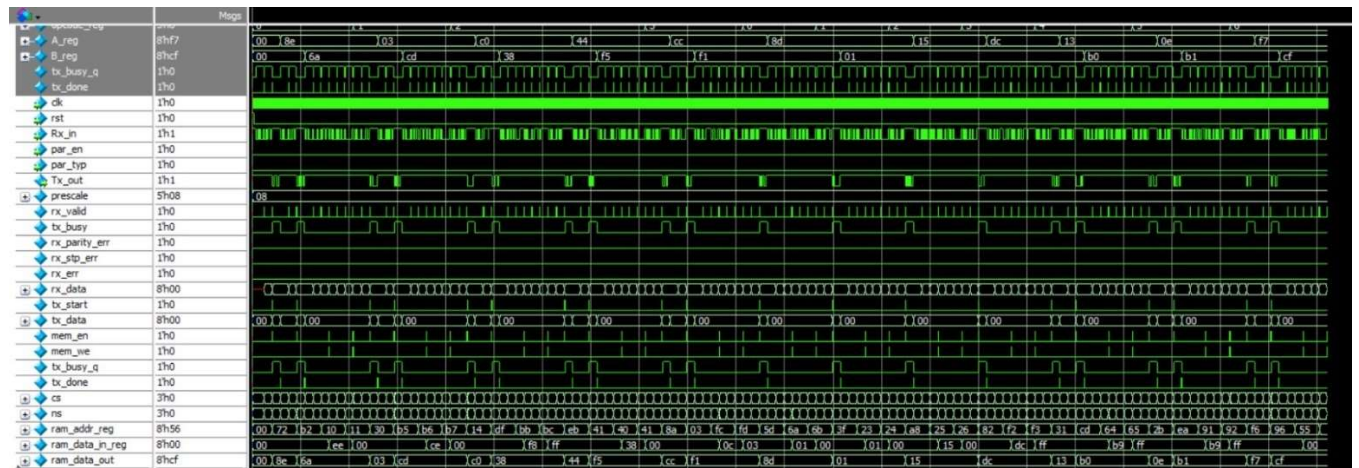
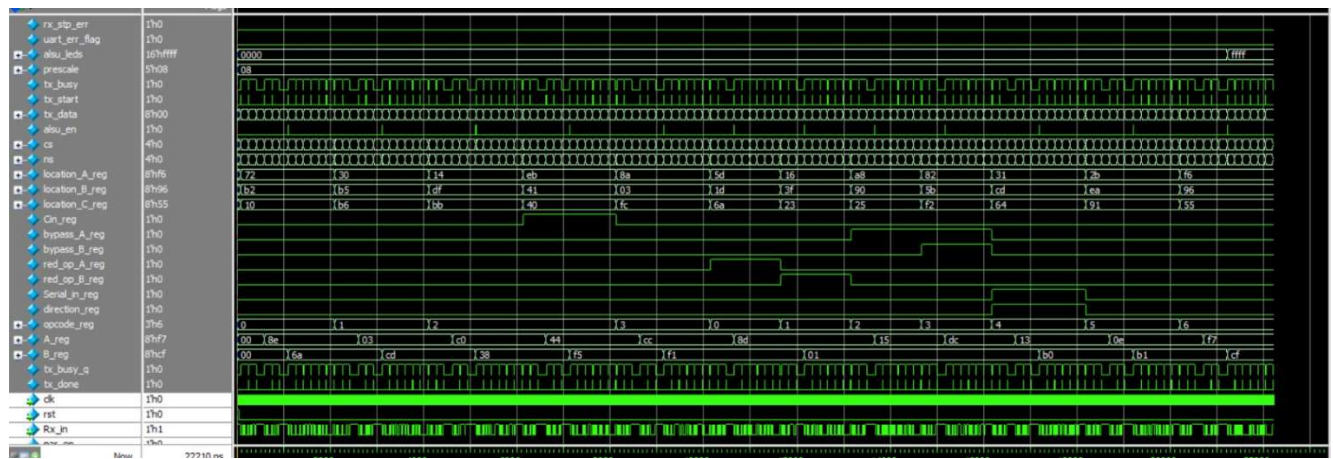
        repeat(2) @(negedge clk);
        Check_result(name);
    endtask
endmodule

```

SIMULATION:

1. Waveform on Questasim:





2. Transcript:

```

# PASS OR A=8e, B=6a, Op=0, C_in_mem=ee, C_Exp=ee
# PASS XOR A=3, B=cd, Op=1, C_in_mem=ce, C_Exp=ce
# PASS ADD A=c0, B=38, Op=10, C_in_mem=fff8, C_Exp=fff8
# PASS ADD Cin A=44, B=f5, Op=10, C_in_mem=38, C_Exp=38
# PASS MULT A=cc, B=f1, Op=11, C_in_mem=30c, C_Exp=30c
# PASS OR Red_A A=8d, B=97, Op=0, C_in_mem=1, C_Exp=1
# PASS XOR Red_B A=31, B=1, Op=1, C_in_mem=1, C_Exp=1
# PASS Bypass A A=15, B=86, Op=10, C_in_mem=15, C_Exp=15
# PASS Bypass A,B A=dc, B=5b, Op=11, C_in_mem=ffdc, C_Exp=ffdc
# PASS Shift Left A=13, B=b0, Op=100, C_in_mem=ffb9, C_Exp=ffb9
# PASS Rotate Rgt A=e, B=b1, Op=101, C_in_mem=ffb9, C_Exp=ffb9
# PASS Invalid Op A=f7, B=cf, Op=110, C_in_mem=0, C_Exp=0
# [FAIL] Invalid Opcode LEDs mismatch!
# Total Passed : 12
# Total Failed : 0
# ** Note: $stop : D:/Digital D&V/projects/UART/Design/TBSS.sv(84)
# Time: 22210 ns Iteration: 1 Instance: /TB
# Break in Module TB at D:/Digital D&V/projects/UART/Design/TBSS.sv line 84

```